

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт перспективных технологий и индустриального программирования

Кафедра индустриального программирования

ОТЧЁТ

по практическому занятию № 2

по дисциплине «Создание программного обеспечения»

Тема: «Анализ требований и планирование разработки монолитного
веб-приложения»

Выполнил:

студент группы ЭФБО-17-24

Васин С. А. _____

Проверил:

Макиевский С. Е. _____

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
Цель работы	3
Задание	3
1 ХОД РАБОТЫ	4
1.1 Основы веб-разработки	4
1.2 Методы сбора и анализа требований	4
1.3 Требования и варианты использования	5
1.4 Планирование ресурсов и этапов	7
1.5 Практический план с зависимостями	8
1.6 Риски и масштабирование	9
1.7 Итоговый план применения	10
2 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ	11
ЗАКЛЮЧЕНИЕ	13
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	13

ВВЕДЕНИЕ

Цель работы

Проанализировать требования GPU Rent, описать варианты использования и составить план монолитного приложения с зависимостями, ресурсами, рисками и возможностями масштабирования.

Задание

1. Изучить основы HTML, CSS, JavaScript, веб-серверов и клиент-серверного взаимодействия.
2. Рассмотреть функциональные и нефункциональные требования, методы их сбора и документирования.
3. Построить варианты использования и оформить требования диаграммой и текстом.
4. Изучить этапы разработки, составление расписания и планирование ресурсов.
5. Составить план монолитного приложения с зависимостями между работами.
6. Рассмотреть управление рисками и будущее масштабирование.
7. Применить результаты к плану разработки GPU Rent.

1 ХОД РАБОТЫ

1.1 Основы веб-разработки

HTML задаёт структуру, CSS — оформление, JavaScript — поведение страницы [1]. Запланирована SPA на React и Vite. HTTP связывает браузер и сервер запросами и ответами [2]: GET /api/v1/catalog/gpus читает каталог, POST /api/v1/instances запускает инстанс. Nginx будет обслуживать TLS и статику, Express — /api; сервер проверяет данные и полномочия, сессии планируются в Redis.

Выбран модульный монолит: auth, catalog, instances, billing, payments, admin. Один исполнитель получает простое развёртывание, а выделение GPU и проверка баланса согласуются транзакцией PostgreSQL [3]. По README реализована только БД; сервер, клиент, Redis и мониторинг запланированы.

1.2 Методы сбора и анализа требований

Анализ включает сбор потребностей, устранение противоречий, спецификацию, проверку и управление изменениями [4]. Функциональные требования описывают действия, нефункциональные — качество; формулировки должны быть проверяемыми.

Применены анализ аналогов из П1 (RunPod, Vast.ai, Immers.cloud), задания и документации, мозговой штурм и прото-персоны студента, разработчика и специалиста по рендерингу. Выделены шаблоны, SSH, баланс и автоостановка. Интервью не проводились, потребности остаются гипотезами.

Требования ведутся в Markdown, диаграммы — PlantUML, версии — Git. MoSCoW разделяет обязательные Must, желательные Should, необязательные Could и исключённые Won't. Идентификаторы FR/NFR связывают функции с модулями и проверками; изменения оцениваются по влиянию на баланс, жизненный цикл и сроки.

1.3 Требования и варианты использования

Таблица 1 — Функциональные требования GPU Rent

ID	Содержание и критерий приёмки	Приоритет
FR-01–02	Регистрация с подтверждением email, вход, выход и смена пароля; до подтверждения запуск запрещён, смена пароля завершает остальные сессии	Must
FR-03	Добавление и удаление SSH-ключей, показ fingerprint; дубликат отклоняется	Must
FR-05–06	Каталог GPU по дата-центрам с ценами и наличием; публичные и собственные приватные шаблоны	Must
FR-07–08	Запуск 1–8 GPU одной ноды из шаблона, проверка баланса на час; статусы pending, running, stopped, terminated, failed; остановка освобождает GPU	Must
FR-10, FR-12	Посекундный учёт, списание раз в минуту по снимку цены; при балансе не выше нуля автоостановка и уведомление	Must
FR-11, FR-13	Пополнение через шлюз, история операций и расходы; повтор уведомления не зачисляет деньги второй раз	Must
FR-14–15	Управление дата-центрами, нодами, GPU и ценами; интервалы цен не пересекаются, изменение не затрагивает снимок запущенного инстанса	Must
FR-09, FR-16–18	Сетевые тома, блокировка пользователей, дашборд загрузки и выручки, аудит значимых действий	Should
FR-04	API-ключи с отзывом и сроком действия; значение показывается один раз, хранится хэш	Could

Итого: 18 FR, 13 Must, четыре Should, одно Could. Учебный контур использует тестовые платежи и эмулятор ноды. Spot отличается только ценой; возобновление требует свободных GPU. Агент устанавливает ключ и сообщает адрес SSH.

Таблица 2 — Нефункциональные требования и проверки

ID	Требование	Проверка или целевой критерий
NFR-01–02	Производительность и доступность	Каталог: p95 < 300 мс при 1000 RPS на стенде 2 vCPU, 4 ГБ; доступность 99,5 % за месяц
NFR-03	Защита сессий, секретов и данных клиентов	TLS, проверка полномочий, RLS, шифрование секретов; отрицательные тесты доступа
NFR-04	Точность денежных операций	numeric; баланс равен сумме журнала, сверочный запрос возвращает ноль расхождений
NFR-05–06	Рост учёта потребления и аудит	Месячные секции usage_records, проверка pruning; аудит только на добавление, хранение не менее года
NFR-07	Удобство и совместимость	Адаптивность от 360 px, две последние версии основных браузеров, Accessibility ≥ 90
NFR-08–09	Воспроизводимость и конфиденциальность	Миграции и тесты на пустой БД; минимизация персональных данных и маскированная аналитика

Нагрузка, доступность и удобство — цели будущих проверок, не измеренные результаты.

Диаграмма вариантов использования: гость и клиент

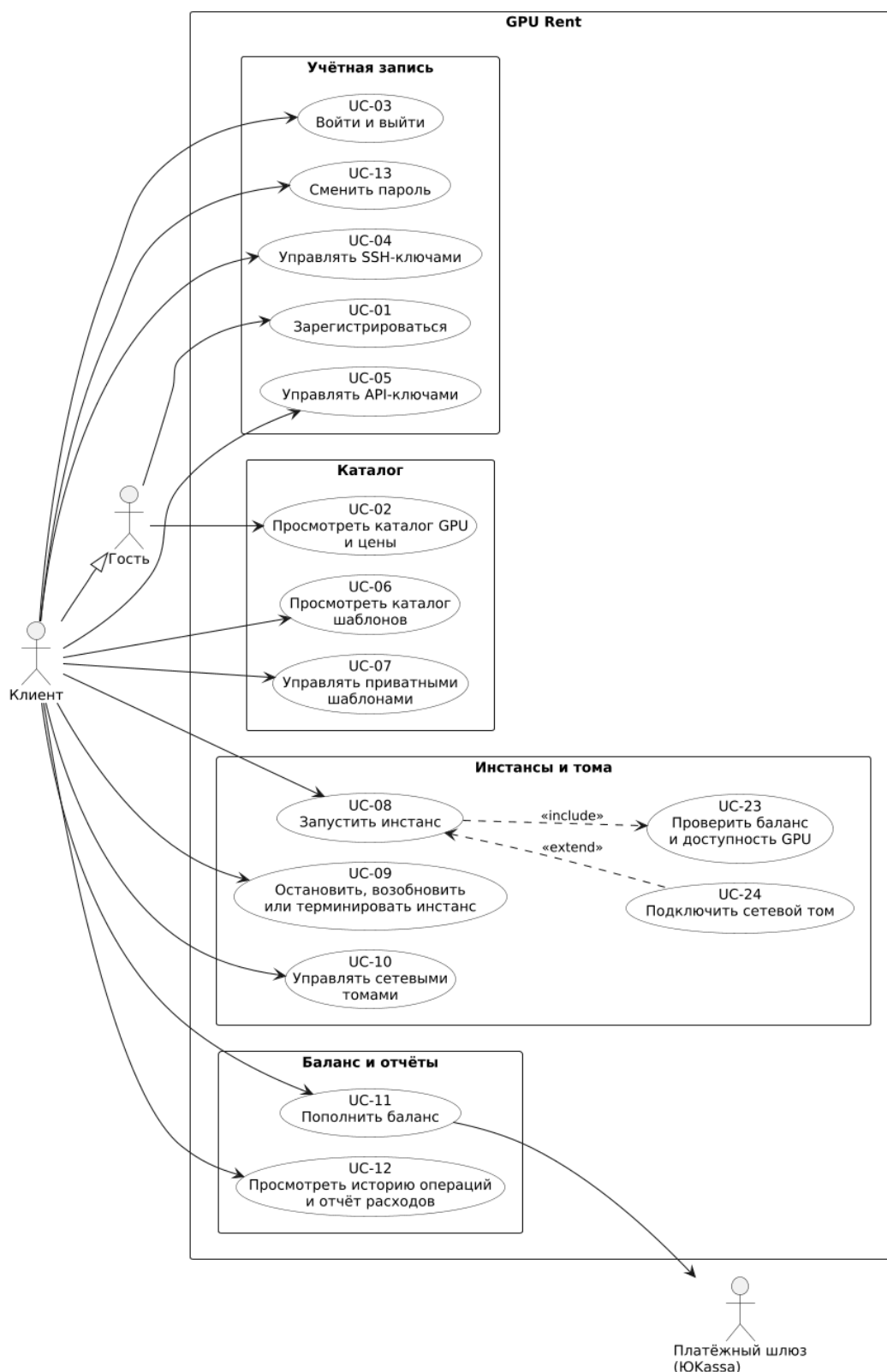


Рисунок 1 — Варианты использования: гость и клиент

Клиент наследует возможности гостя; запуск включает проверку ресурсов (*include*), том расширяет сценарий (*extend*). Дополнительные акторы: администратор, планировщик биллинга и агент ноды.

UC-08 «Запустить инстанс». Актор — клиент; цель — получить GPU для вычислений. Предусловия: подтверждённая учётная запись, выбранный SSH-ключ, действующий шаблон и достаточный баланс. Основной сценарий:

1. Клиент выбирает дата-центр, модель, количество GPU, тариф, шаблон и диск.
2. Сервер проверяет права, входные данные, баланс и свободные GPU одной ноды.
3. Транзакция резервирует GPU, фиксирует цену и создаёт инстанс *pending*.
4. Агент получает команду запуска и сообщает результат.
5. После *running* клиент получает параметры SSH-подключения.

Нехватка средств или GPU отклоняет запрос; отказ агента приводит к *failed* и освобождению ресурса. Постусловие: GPU закреплены только за клиентом. Трассировка: FR-03, FR-07–08 → UC-08 → *instances* → *fn_start_instance*; проверка — конкурентный запуск.

1.4 Планирование ресурсов и этапов

WBS объединяет проект в анализ, реализацию и выпуск, включающие девять блоков таблицы 3. Один исполнитель совмещает все роли; плановая ёмкость — 8–10 часов в неделю. Ресурсы: Git, PostgreSQL, PlantUML; будущий стенд — VPS, Compose, Nginx и Redis.

Оценки исходного плана рассчитаны по трём точкам: $E = (O + 4M + P) / 6$. Для недель 1–14 получено 119,82 часа; это план, не фактический хронометраж. При перегрузке сокращаются *Should* и *Could*.

1.5 Практический план с зависимостями

Таблица 3 — Укрупнённый план GPU Rent (2026 год)

Блок, недели	Результат и критерий готовности	Зависимость
1. Введение, 1	Тема, роли, репозиторий и процесс; сформулирован MVP	—
2. Анализ, 2	FR/NFR, прецеденты, план и риски согласованы	Блок 1
3. База данных, 3–4	Логическая модель, миграции, функции, seed, тесты и замеры	Требования
4. Web-прототип, 5–6	Каркас, auth, catalog, SPA входа и каталога; сценарии проверены	Схема и функции БД
5. Окружение, 7–8	Compose, TLS, CI/CD, копии; восстановление проверено	Прототип
6. Анализ данных, 9	Объёмы, сроки хранения, ревизия требований	Тестовые данные
7. БД, итерация 2, 10	Секции и индексы уточнены, миграции и тесты проходят	Анализ данных
8. API, 11–12	OpenAPI, instances, billing, payments, admin; тест повторного платежа	Контракт API и БД
9. Выпуск, 13–14	Релиз 0.9, две реплики, логи и нагрузочные проверки	API, CI/CD, TLS

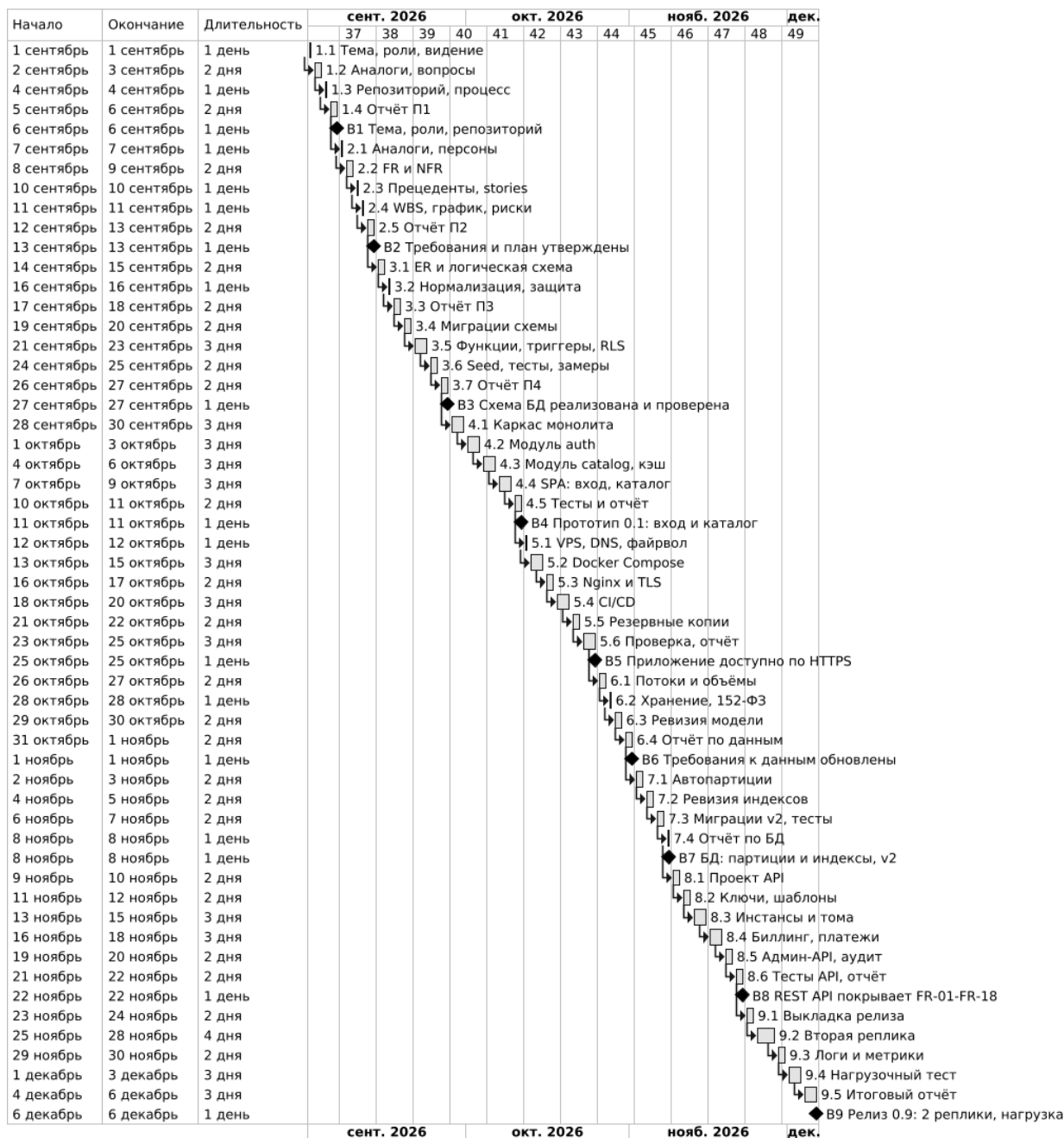


Рисунок 2 — График работ GPU Rent, недели 1–14

Плановые вехи: 13.09 — требования и план; 27.09 — БД; 11.10 — прототип; 25.10 — HTTPS-стенд; 22.11 — API; 06.12 — релиз. Критическая цепочка: требования → модель → функции БД → анализ данных → API → релиз. Задержка последовательной работы сдвигает выпуск.

1.6 Риски и масштабирование

Приоритет риска — вероятность × влияние (шкалы 1–5). Стратегии: избежать, снизить, передать, принять; пересмотр после итерации.

Таблица 4 — Основные риски и меры

Риск	Мера и проверка
Перегрузка исполнителя, недооценка	Ограничить текущие задачи, сравнивать план и факт; при отставании более 20 % сокращать необязательный объём
Двойное выделение GPU	Ограничение исключения и блокировки БД; тест параллельного запуска
Ошибка баланса, повтор платежа	Неизменяемый журнал, <code>numeric</code> , идемпотентное зачисление; сверка и повтор <code>webhook</code>
Утечка данных или потеря БД	Проверки доступа, RLS, шифрование, резервные копии и проверка восстановления

Масштабирование следует измерениям [5, 6]. p95 каталога > 300 мс три пятиминутных окна — проверить запросы и кэш; CPU > 70 % в течение 15 минут — рассмотреть ресурсы и реплики; биллинг > 30 секунд — отдельный worker с защитой от двойного запуска. Мониторинг и эти меры запланированы.

1.7 Итоговый план применения

MVP проверяется сквозным сценарием «регистрация → пополнение → запуск → SSH → списание → остановка», изоляцией клиентов и повторными запросами. Недели 15–32 отведены полному интерфейсу, безопасности и эксплуатации. До реального запуска нужно уточнить подтверждение остановки агентом, допустимый долг, историю размеров томов и округление списаний.

2 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Какие основы веб-разработки были изучены? HTML, CSS, JavaScript, HTTP, клиент-серверная модель, Nginx, Express и модульный монолит.

Сколько времени было уделено изучению темы? Фактический хронометраж не фиксировался. Плановые оценки работ нельзя выдавать за затраченное время.

В чем заключается процесс анализа требований? В выявлении потребностей, устранении противоречий, записи проверяемых требований, согласовании и управлении изменениями.

Какие ошибки или недоразумения могут возникнуть при анализе требований? Неизмеримые формулировки, пропущенные исключения и неподтверждённые гипотезы. Их выявляют сценарии и проверка источников.

Какие ключевые аспекты веб-разработки были изучены? HTTP, сессии, безопасность, API и транзакции; согласованность денег и выделения GPU.

Могли бы вы рассказать о методах сбора требований? Используются интервью, анкеты, наблюдение, анализ документов и аналогов, сценарии и прототипы. Здесь применены документы, аналоги и прото-персоны; прямые исследования пользователей ещё не проводились.

Какие требования вы можете собрать и документировать для веб-приложения? Функции, качество, ограничения и внешние интерфейсы. В GPU Rent это запуск, оплата и управление, а также скорость, безопасность и точность баланса.

Какие методы управления рисками и масштабирования были рассмотрены? Оценка вероятности и влияния, выбор стратегии и регулярный пересмотр. Для нагрузки — вертикальное увеличение ресурсов, кэш, реплики, секционирование и фоновые обработчики.

Можете ли вы объяснить основные концепции веб-разработки? Клиент отправляет запрос, сервер проверяет права и данные, выполняет бизнес-операцию и возвращает ответ. При запуске GPU транзакция согласует баланс и резервирование, затем агент сообщает состояние.

Какие методы анализа требований вы применяли в учебных упражнениях? Анализ аналогов и документов, прото-персоны, мозговой штурм, MoSCoW, прецеденты и критерии приёмки. Требования связаны идентификаторами с модулями и проверками.

Какие этапы разработки веб-приложения были включены в ваш практический план? Анализ, проектирование БД, web-прототип, окружение, уточнение данных, API и выпуск. Полный интерфейс и эксплуатация детализируются после первых 14 недель.

Какие решения и меры были предложены для управления рисками и масштабирования? Защита инвариантов БД, идемпотентность, проверки доступа, копии и сокращение необязательных задач. Масштабирование привязано к порогам задержек и загрузки.

Можете ли вы детально объяснить весь процесс веб-разработки? Требования переходят в модель данных, контракт API и экраны, затем в код и тесты. После проверки выполняются развёртывание, наблюдение за работой, сбор обратной связи и следующая итерация.

Какие конкретные методы и инструменты использовались при анализе требований? MoSCoW, сценарии и таблицы FR/NFR; Markdown, PlantUML и Git. Источниками служили задание, аналоги и документация.

Какие ключевые этапы и зависимости были учтены при создании практического плана разработки? Требования предшествуют схеме, схема — функциям и API, проверенный API — выпуску. CI/CD и TLS должны быть готовы до выкладки, анализ данных — до ревизии секций.

Какие стратегии и решения были предложены для учета рисков и масштабирования в будущем? Регулярно пересматривать риски и оценки, сохранять модульные границы и измерять нагрузку. Кэш, реплики и worker вводить по подтверждённому узкому месту, уточнив эксплуатационные ограничения.

ЗАКЛЮЧЕНИЕ

Для GPU Rent описаны 18 FR, девять NFR, сценарий запуска и план с ресурсами, зависимостями и рисками. Выбран модульный монолит с измеримыми триггерами масштабирования. БД реализована; веб-часть, валидация пользовательских гипотез и нагрузочные проверки запланированы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. MDN Web Docs: Веб-технологии для разработчиков. — URL: <https://developer.mozilla.org/ru/docs/Web> (дата обращения: 30.09.2026).
2. Fielding R., Nottingham M., Reschke J. RFC 9110: HTTP Semantics. — IETF, 2022. — URL: <https://www.rfc-editor.org/rfc/rfc9110> (дата обращения: 30.09.2026).
3. Fowler M. MonolithFirst. — 2015. — URL: <https://martinfowler.com/bliki/MonolithFirst.html> (дата обращения: 30.09.2026).
4. Вигерс К., Битти Дж. Разработка требований к программному обеспечению. — 3-е изд. — СПб.: БХВ-Петербург, 2014.
5. PostgreSQL 16 Documentation. — URL: <https://www.postgresql.org/docs/16/> (дата обращения: 30.09.2026).
6. Клеппман М. Высоконагруженные приложения. Программирование, масштабирование, поддержка. — СПб.: Питер, 2018.